

LATIHAN SOAL

Soal No. 1

function countPos(l: List) → integer

{ Menghitung banyaknya kemunculan bilangan positif (> 0) dari sebuah list of integer l }

KAMUS LOKAL

type Address: < info: integer, next: Address >

temp: Address

count: integer

ALGORITMA

count $\leftarrow$ 0

temp $\leftarrow$ l

while (temp $\neq$ NIL) do

if (temp↑.info $>$ 0) then

 count $\leftarrow$ count + 1

 temp $\leftarrow$ temp↑.next

→ count

LATIHAN SOAL

Soal No. 2

function max(l: List) → integer

{ Menghasilkan nilai maksimum dari suatu list of integer l yang tidak kosong }

KAMUS LOKAL

type Address: < info: integer, next: Address >

max: integer

p: Address

ALGORITMA

p $\leftarrow$ l

max $\leftarrow$ p↑.info { list pasti tidak kosong }

p $\leftarrow$ p↑.next

while (p $\neq$ NIL) do

if (p↑.info $>$ max) then

 max $\leftarrow$ p↑.info

 p $\leftarrow$ p↑.next

→ max

LATIHAN SOAL

Soal No. 3

function searchPos(l: List) → address

{ Menghasilkan address di mana nilai positif pertama kali ditemukan di list of integer l }

KAMUS LOKAL

type Address: < info: integer, next: Address >
p: Address

ALGORITMA

p ← l
while (p ≠ NIL and p↑.info ≤ 0) do
 p ← p↑.next
→ p

LATIHAN SOAL

Soal No. 4

pocedure deleteNeg(input/output l: List)

{ Menghapus semua elemen bernilai negatif (< 0) pada sebuah list of integer l. List l boleh kosong dan setiap elemen yang dihapus harus dilakukan dealokasi. }

KAMUS LOKAL

type Address: < info: integer, next: Address >
del, curr, prev: Address

ALGORITMA

curr ← l
prev ← NIL
while (curr ≠ NIL) do
 if (curr↑.info < 0) then
 del ← curr { agar dapat di-dealokasi tanpa
 memengaruhi curr }
 curr ← curr↑.next
 if (prev = NIL) then { delete first }
 l ← curr
 else { delete last/middle }

```
        prev↑.next ← curr
    dealokasi(del)
else
    prev ← curr
    curr ← curr↑.next
```

LATIHAN SOAL

Soal No. 5

```
procedure copyPos(input l1: List, output l2: List)
{ Menyalin semua elemen bernilai positif (> 0) dari sebuah list
  of integer l1 menjadi l2 }
```

KAMUS LOKAL

```
type Address: < info: integer, next: Address >
p, prev: Address
node: Address
```

ALGORITMA

```
l2 ← NIL
p ← l1
prev ← NIL { mencari prev di l1 }
while (p ≠ NIL) do
    if (p↑.info > 0) then
        alokasi (node)
        if (node ≠ NIL) then
            node↑.info ← p↑.info
            node↑.next ← NIL
            if (l2 = NIL) then { l2 masih kosong }
                l2 ← node
            else
                prev↑.next ← node
            prev ← node
        p ← p↑.next
```

LATIHAN SOAL

Soal No. 6

procedure sortedInsert(input/output l: List, input x: ElType)
{ Menambahkan sebuah elemen x pada sebuah list of integer l
yang terurut menaik }

KAMUS LOKAL

type Address: < info: integer, next: Address >
node, prev, curr, p: Address

ALGORITMA { Alternatif 1 }

alokasi (node)

if (node ≠ NIL) then

node↑.info ← x

node↑.next ← NIL

if (l = NIL or x ≤ l↑.info) then { insert first }

node↑.next ← l

l ← node

else

curr ← l

prev ← NIL

while (curr ≠ NIL and curr↑.info < x) do

prev ← curr

curr ← curr↑.next

prev↑.next ← node

node↑.next ← curr

ALGORITMA { Alternatif 2 }

alokasi (node)

if (node ≠ NIL) then

node↑.info ← x

node↑.next ← NIL

if (l = NIL or x ≤ l↑.info) then

node↑.next ← l

l ← node

else

p ← l

while (p↑.next ≠ NIL and p↑.next↑.info < x) do

{ sudah pasti tidak NIL karena sudah di cek
sebelumnya }

```
p ← p↑.next  
node↑.next ← p↑.next  
p↑.next ← node
```

LATIHAN SOAL

Soal No. 7

```
procedure updateList (input x, y: ElType, input/output l: List)  
{ Menerima sebuah infotype x dan y dan sebuah list l dan  
  kemudian mengganti elemen pertama l yang bernilai x dengan y  
  jika x ada di l. Jika x tidak ada di l, l tetap. }
```

KAMUS LOKAL

```
type Address: < info: integer, next: Address >  
p: Address
```

ALGORITMA

```
p ← l  
while (p ≠ NIL and p↑.info ≠ x) do  
  p ← p↑.next  
if (p ≠ NIL) then  
  p↑.info ← y
```